Kai Battistoni

10/26/2026

Software Requirements Specification: Local LLM Autonomous Player Combat Agent for Minecraft 26.3

1. Purpose and Scope

Purpose and Goal

Standard Minecraft combat against autonomous or simulated player companions lacks realistic, human-like tactical decision-making unless controlled by complex hard-coded rulesets. Minecraft players seeking AI companions/opponents, mod developers testing combat balance, and researchers evaluating local agent intelligence lack an autonomous player-character mod that executes smart tactics while strictly respecting vanilla survival game rules. The goal of this project is to deliver provide a combat overhaul to the locally executed LLM mod for Minecraft 26.3 that autonomously drives a Player Character to engage in tactical combat using strict vanilla player capabilities without requiring external cloud API dependencies.

In Scope

- Reconfigure/add features to existing local LLM runtime integration (via Ollama local server) executing decisions asynchronously from the main game render/logic loop.
- Camera aiming, weapon selection, and combat action dispatching.
- Target-specific tactical combat routines (e.g., digging/placing 1x2 block overhang shelters against Endermen, hit-and-retreat kiting against Creepers, tactical water positioning).
- Strict enforcement of vanilla player combat constraints: 4.5-block interaction reach limit, attack cooldown timers, and item management.
- Developer HUD overlay and logging tools to track target selection, prompt context, and selected tactical states in real time.

Out of Scope

- Cloud-based LLM APIs (OpenAI, Anthropic, etc.).

- Modifying hostile mob pathfinding or mob AI (mobs utilize standard vanilla behavior; only the player character is LLM-controlled).
- Bypassing vanilla game mechanics (e.g., flight, teleportation, infinite reach, or instant block breaking).
- Cross-platform mobile or console support (PC Java Edition on Java 25 / Fabric runtime only).
- Procedural generation of custom 3D models or combat animations via LLM.

2. Stakeholders and Users

- Player: Needs an engaging, human-like AI player companion or opponent that reacts dynamically to hostiles using realistic survival tactics without breaking game rules or dropping frame rates.
- Server Admin: Needs administrative toggle switches and resource management controls (CPU/GPU thread limits) to prevent local inference tasks from lagging the Minecraft tick rate (20 TPS).
- Grader / Instructor: Needs clean section ordering, traceable requirement IDs, unambiguous functional rules, and deterministic verification methods for evaluating the assignment.
- Target Hardware / Device: Needs a strictly bounded memory footprint (≤ 7.75 GB VRAM overhead) and low CPU utilization so local inference runs smoothly alongside Java 25.
- Family Member / Spectator: Provide greater visual feedback explaining the agent's current target and tactical plan.

3. Context, Constraints and Assumptions

Technical Context & Constraints

- Language: Java 25 (Minecraft 26.3 modding standard)
- Platform: Windows, Linux, and macOS desktop (Java Edition).
- AI Engine: Gemma model weights executed via a locally hosted Ollama server instance.
- Navigation & Mining Engine: Baritone API for pathfinding, movement execution, and automated mining.
- Deadline: 10/29/2026
- Team Size: 1 developer (For this component).

Assumptions and Dependencies

- [Assumption 1 - Confirmed by Team]: The local Ollama server running Gemma responds to local HTTP/JSON requests within 100 ms under standard combat loads.

- [Assumption 2 - Confirmed by Team]: Baritone API can be successfully invoked programmatically from Fabric event listeners to execute high-level tactical waypoints (e.g., Baritone.getProvide().getPathingBehavior().cancelEverything()).
- [Assumption 3 - Needs Confirmation]: Baritone's block-breaking algorithms respect survival minin**A**g speeds and do not grant the agent illegal excavation advantages during combat.
- Dependencies: Fabric API (Minecraft 26.3), Java 25 JRE, local Ollama server binary running Gemma, Baritone API library.

4. Functional Requirements

ID	Requirement	Priority	Traces to	Verified by
FR-1	The mod must asynchronously serialize local environment state (player health, hotbar items, target mob type, distance, Baritone status) and POST it to the local Ollama server running Gemma without blocking the main game render loop.	Must	Goal (Local AI integration) & Player	test: ollama_async_push_no_block
FR-2	The mod must translate high-level Gemma tactical outputs (e.g., retreat, approach, mine cover) into direct API calls for Baritone to execute pathfinding, navigation, and emergency mining.	Must	Goal (Baritone integration) & Player	test: baritone_waypoint_execution
FR-3	All player melee attack actions must strictly enforce the vanilla 4.5-block reach limit and weapon-specific attack cooldown meters before issuing an attack tick.	Must	Goal (vanilla limits) & Player (fair survival rules)	test: enforce_reach_and_cooldown
FR-4A	When fighting Spiders or Endermen, Gemma must command Baritone to navigate to elevated blocks, create a 1x2 overhang shelter, or path into nearby water.	Must	Goal (mob-specific tactics: Spiders/Endermen)	test: spider_enderman_terrain_tactics

ID	Requirement	Priority	Traces to	Verified by
FR-5A	When fighting Shulkers or Wardens, Gemma must direct Baritone to utilize line-of-sight obstruction pillars to block Shulker bullet tracking and Warden sonic booms.	Must	Goal (mob-specific tactics: Shulkers/Wardens)	test: shulker_warden_cover_tactics
FR-6A	When fighting Skeletons or Witches, the agent must raise a shield (if available) against incoming skeleton arrows while Baritone closes the distance, and aggressively rush witches before splash potion volleys.	Must	Goal (mob-specific tactics: Skeletons/Witches)	test: skeleton_witch_rush_tactics
FR-7A	When encountering Piglins, Piglin Brutes, or Hoglins, the agent must equip Gold Armor to neutralize Piglin hostility, kite Piglin Brutes with hit-and-retreat movement, and maintain distance from Hoglin charge vectors.	Must	Goal (mob-specific tactics: Nether mobs)	test: nether_mob_tactics
FR-8A	When facing Blazes or Ghasts, the agent must use Baritone to strafe behind cover to dodge fireball volleys and time melee/projectile counter-attacks.	Must	Goal (mob-specific tactics: Blazes/Ghasts)	test: blaze_ghast_strafe_tactics
FR-9A	When fighting Pillagers, Vindicators, or Ravagers during raids, the agent must prioritize intercepting ranged crossfire with shields and targeting evokers/pillagers first via Baritone positioning.	Must	Goal (mob-specific tactics: Raid mobs)	test: raid_mob_prioritization
FR-10A	If the Ollama server times out (>100 ms) or returns malformed JSON, the agent must fall back to a deterministic	Must	Player (low latency) & Reliability	test: ollama_timeout_fall

ID	Requirement	Priority	Traces to	Verified by
	defensive state (raising shield and holding position) while waiting for response / trying again.			back
FR-11A	Server administrators must be able to configure the Ollama server URL, port, and model used via JVM launch arguments	Should	Server Admin (configuration flexibility)	test: admin_ollama_endpoint_config
FR-12	The mod must log all invalid Gemma JSON payloads, connection drops, and 1-based prompt line numbers to logs/gemma_agent.log.	Should	Grader & Server Admin (debugging)	test: log_malformed_gemma_output
FR-13A	Players must be able to toggle an in-game agent debug overlay showing active Gemma tactical states and Baritone path status using the F8 keybind.	Could	Family Member / Spectator & Player	test: toggle_agent_debug_hud
FR-14A	Upon tracking a nearby mob, it must not immediately stop other actionable tasks, unless 1. The player takes damage from a mob, or 2. The mob is tracking the player (Not wandering / idle)	Must	Target-specific tactical combat routines	

5. Non-Functional Requirements

ID	Quality	Condition and Measurement	Verified by
NFR-1	Performance	95% of local Ollama HTTP requests return valid Gemma tactical decisions in ≤ 100 ms on standard test hardware (AMD Ryzen 5 7600X, 32GB RAM, RTX 3070) across 50 warm combat cycles.	Automated benchmark harness measuring HTTP round-trip timing.
NFR-2	Reliability	The autonomous agent must execute continuous combat loops against mixed mob waves (including Nether and Raid mobs) for 2 hours with 0 unhandled exceptions or memory leaks.	Continuous automated stress-test script running in headless mode for 2 hours.
NFR-3	Usability / Accessibility	All agent status overlays and HUD text elements must maintain a minimum contrast ratio of 4.5:1 against variable world lighting and respect text scale settings.	Manual WCAG 2.1 contrast inspection tool under bright daylight and pitch-black cave render tests.

6. User Stories

Story 1: Skeleton Shielding & Witch Rushing

- As a player,
- I want the AI agent to raise its shield (If one is available) against Skeletons while closing distance, and aggressively rush Witches to interrupt potion attacks,
- so that ranged mobs are countered efficiently using correct tactical responses.
- Covers: FR-3, FR-6, FR-10

  Acceptance Criteria:

- Given a Skeleton targets the agent from 10 blocks away,

  When an arrow is loosed,

Then the agent equips and raises a shield while Baritone paths forward to close melee range.

- Given a Witch begins drinking a splash potion of harming within 8 blocks,

  When Gemma processes the encounter,

  Then the agent cancels defensive pathing and executes an immediate sprint-attack vector to interrupt the Witch.

Story 2: Baritone-Powered Nether Mob Management (Piglins, Brutes, Hoglins)

- As a player,
- I want the agent to manage Nether hostiles by verifying gold armor against Piglins, kiting Brutes, and evading Hoglin charges,
- so that high-damage Nether encounters are survived without breaking survival rules.
- Covers: FR-2, FR-7

  Acceptance Criteria:

- Given the agent enters a Bastion remnant without any gold armor,

  When Piglins detect the agent,

  Then Gemma triggers an immediate inventory swap to equip Gold Boots, resetting Piglin hostility.

- Given a Hoglin initiates a charge attack animation,

  When the tactical tick evaluates,

  Then Gemma commands Baritone to sidestep the charge vector by 3 blocks while attempting to counter-attack.

Story 3: Warden & Shulker Cover Utilization

- As a player,
- I want the agent to utilize Baritone to take cover behind blocks when fighting Wardens and Shulkers,
- so that devastating ranged attacks (sonic booms and levitation bullets) are blocked by terrain.
- Covers: FR-2, FR-5

  Acceptance Criteria:

- Given a Shulker fires a levitation bullet at the agent,

  When the projectile is tracked,

Then Baritone immediately paths the agent behind the nearest solid column or wall to intercept the bullet.

- Given a Warden charges its sonic boom attack,

  When the warning sound triggers,

  Then the agent instantly breaks line-of-sight using Baritone obstacle pathing.

- Given a Warden/Skulk Shriekers is/are detected in the nearby vicinity, command Baritone to sneak while walking, a give directive to leave the deep-dark biome, travel some distance > 200 blocks before continuing original plan.

Story 4: Ollama Timeout Recovery & Malformed Logging

- As a grader,
- I want connection timeouts or malformed Gemma outputs logged with 1-based line numbers and handled safely,
- so that system stability and error recovery can be verified.
- Covers: FR-10, FR-12

  Acceptance Criteria:

- Given the local Ollama server fails to respond within 100 ms,

  When the timeout threshold is reached,

  Then the agent drops active Baritone pathing, raises its shield, and logs a warning to logs/gemma_agent.log.

- Given Ollama returns corrupt JSON syntax,

  When the parser fails,

  Then the 1-based line number and error are written to the log file without crashing the client thread.

7. Traceability Matrix

Goal	User Need	Requirement IDs	User Stories	How Verified
Deliver Local Gemma/ Baritone	Player (adaptive tactical combat	FR-1, FR-2, FR-3, FR-4, FR-5, FR-6, FR-7, FR-8,	Story 1, Story 2,	test: ollama_async_push_no_block, test: baritone_waypoint_executio

Goal	User Need	Requirement IDs	User Stories	How Verified
Agent	across diverse mob types)	FR-9	Story 3	n, test: spider_enderman_terrain_tactics, test: nether_mob_tactics, test: shulker_warden_cover_tactics
Deliver Local Gemma/ Baritone Agent	Server Admin (endpoint config & error tracking)	FR-11, FR-12	Story 4	test: admin_ollama_endpoint_config, test: log_malformed_gemma_output
Deliver Local Gemma/ Baritone Agent	Grader / Instructor (traceability & error logging)	FR-10, FR-12, NFR-2, NFR-4	Story 4	Automated test suite & SonarQube analysis checks
Deliver Local Gemma/ Baritone Agent	Target Device (bounded overhead, performance)	FR-1, FR-10, NFR-1, NFR-2	Story 4	Benchmark harness & 2-hour stress-test script
Deliver Local Gemma/ Baritone Agent	Family Member / Spectator & Player (visual feedback)	FR-13, NFR-3	Story 1, Story 3	test: toggle_agent_debug_hud, WCAG contrast inspection

8. Open Questions

1. Question: How does Baritone handle rapid waypoint cancellation when Gemma rapidly switches tactical intents (e.g., switching from retreating from a Hoglin to rushing a Witch)?

- Who can answer: Lead Software Architect / Baritone API integration specialist.
- Assumed in the meantime: We assume invoking Baritone's path cancellation queue every combat tick prevents pathing lockups, with a minor performance overhead.

2. Question: Will running a local Ollama server hosting Gemma on the same desktop hardware as Minecraft 26.3 cause frame drops on minimum student spec machines during intense raid events?

- Who can answer: Instructor / Grader regarding target hardware performance benchmarks.
- Assumed in the meantime: We assume Gemma running on Ollama utilizes ≤ 7.75 GB VRAM/RAM, leaving sufficient resources for Minecraft and Baritone.

3. Question: Should complex multi-mob scenarios (such as a Woodland Mansion raid with Pillagers and Vexes) be bundled into a single compressed JSON prompt or handled via hierarchical sub-prompts?

- Who can answer: Teammate handling Gemma prompt engineering.
- Assumed in the meantime: We assume a structured JSON prompt summarizing the top 3 highest-threat nearby entities within a 16-block radius keeps token counts and latency within the 100 ms limit.

4. Question: Does the newly implemented command: `/llmagent config endpoint <url>`, utilize a secondary Ollama chat instance, and if so, does that negatively impact performance?

- Who can answer: Developer running tests or LLM Agent with Repo access
- Assumed in the meantime: We assume it is a separate instance, and there is potential for performance improvements if deemed necessary in the future, but existing knowledge already showed LLM responses meeting the 100ms goal.

Feature: Advanced Combat Intelligence & Horde Triage

Use Case: Autonomous Combat Threat Resolution & Multi-Mob Triage

- **Actor:** Autonomous Player Agent (controlled by Gemma via Ollama and Baritone)
- **Precondition:** The player agent is in survival mode, equipped with standard gear (iron armor, weapons, shield), and a hostile mob horde or high-threat special mob (e.g., Warden, Piglin Brute, Witch, or mixed raid wave) enters aggro range 16 blocks.
- **Trigger:** Environmental sensor telemetry detects multiple hostile entities or a high-threat mob, firing an immediate threat-prioritization event tick.

Main Flow:

1. The agent's sensory buffer scans all entities within a 16-block radius, calculating threat scores (Warden/Brute/Ranged > Melee).
2. The mod formats and POSTs prioritized target telemetry JSON to the locally hosted Ollama server running Gemma.
3. Gemma processes the telemetry and returns a structured JSON tactical command (e.g., PRIMARY_TARGET: WANDERING_WITCH, TACTIC: RUSH_INTERRUPT, SECONDARY_PATH: STRAFE_LEFT).
4. The mod parses Gemma's JSON decision payload successfully within 100 ms.
5. The mod dispatches Baritone waypoint and combat action commands (raising shield, pathing to melee range, striking on weapon cooldown).
6. The agent successfully executes the combat maneuver, eliminating or evading the high-threat target.

Alternates:

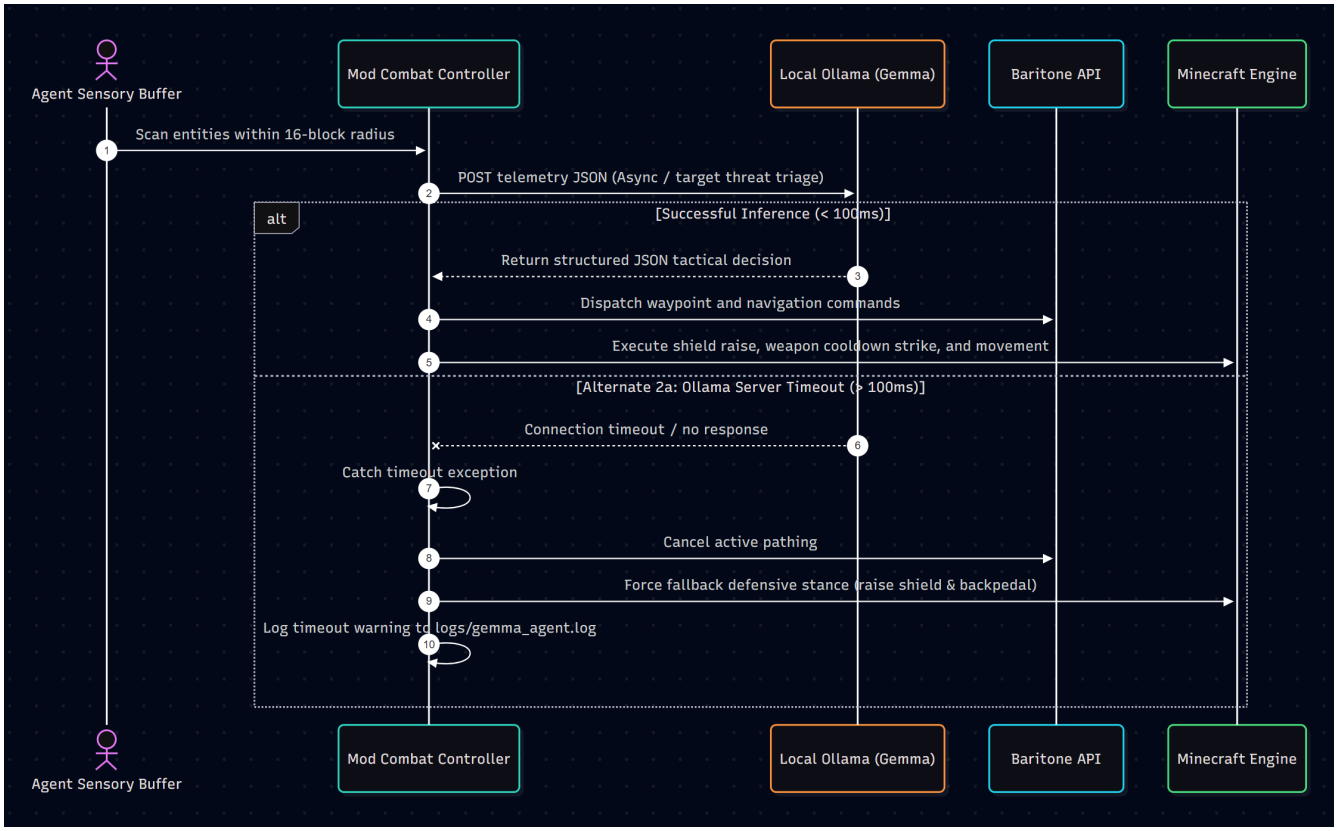
- **2a. Ollama Server Connection Timeout:**
- 2a1. The local HTTP request to Ollama exceeds the 100 ms threshold or fails to connect.
- 2a2. The mod catches the timeout exception, drops active Baritone pathing, forces a deterministic defensive stance (raising shield and backing up), and logs the timeout event to logs/gemma_agent.log.
- **3a. Malformed JSON Response from Gemma:**
- 3a1. Gemma returns corrupt or incomplete JSON syntax due to horde token congestion.
- 3a2. The parser catches the syntax exception, logs the 1-based prompt line number, and triggers an emergency fallback retreat vector.
- **4a. Baritone Pathfinding Obstruction (Dead End / Trap):**

- 4a1. Baritone encounters an impassable wall, lava, or corner trap while executing Gemma's tactical route.
- 4a2. Baritone reports a path failure status; the mod overrides the route and triggers a local emergency pillaring/digging macro to secure high ground.
- **5a. High-Threat Horde Overload (Conflicting Targets):**
- 5a1. More than 8 hostile entities surround the agent simultaneously, flooding the context window with competing threats.
- 5a2. The mod filters and compresses lower-tier mobs into a single aggregate threat token before dispatching the payload to prevent model confusion.
- **Postcondition:** The agent successfully neutralizes or escapes the immediate combat threat without exceeding the 7.75 GB memory overhead or dropping below zero health, verified by the test assertion:

  assert agent.isInCombat() == false || agent.getHealth() > 0;

Sequence Diagram: Advanced Combat Intelligence & Horde Triage

(Note: The sequence diagram below illustrates the main flow of telemetry transmission, Gemma decision-making, and Baritone execution, alongside Alternate 2a for server timeout recovery.)



Appendix

p.13 - Combat Controller Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Agent as Agent Sensory Buffer
    participant Mod as Mod Combat Controller
    participant Ollama as Local Ollama (Gemma)
    participant Baritone as Baritone API
    participant MC as Minecraft Engine

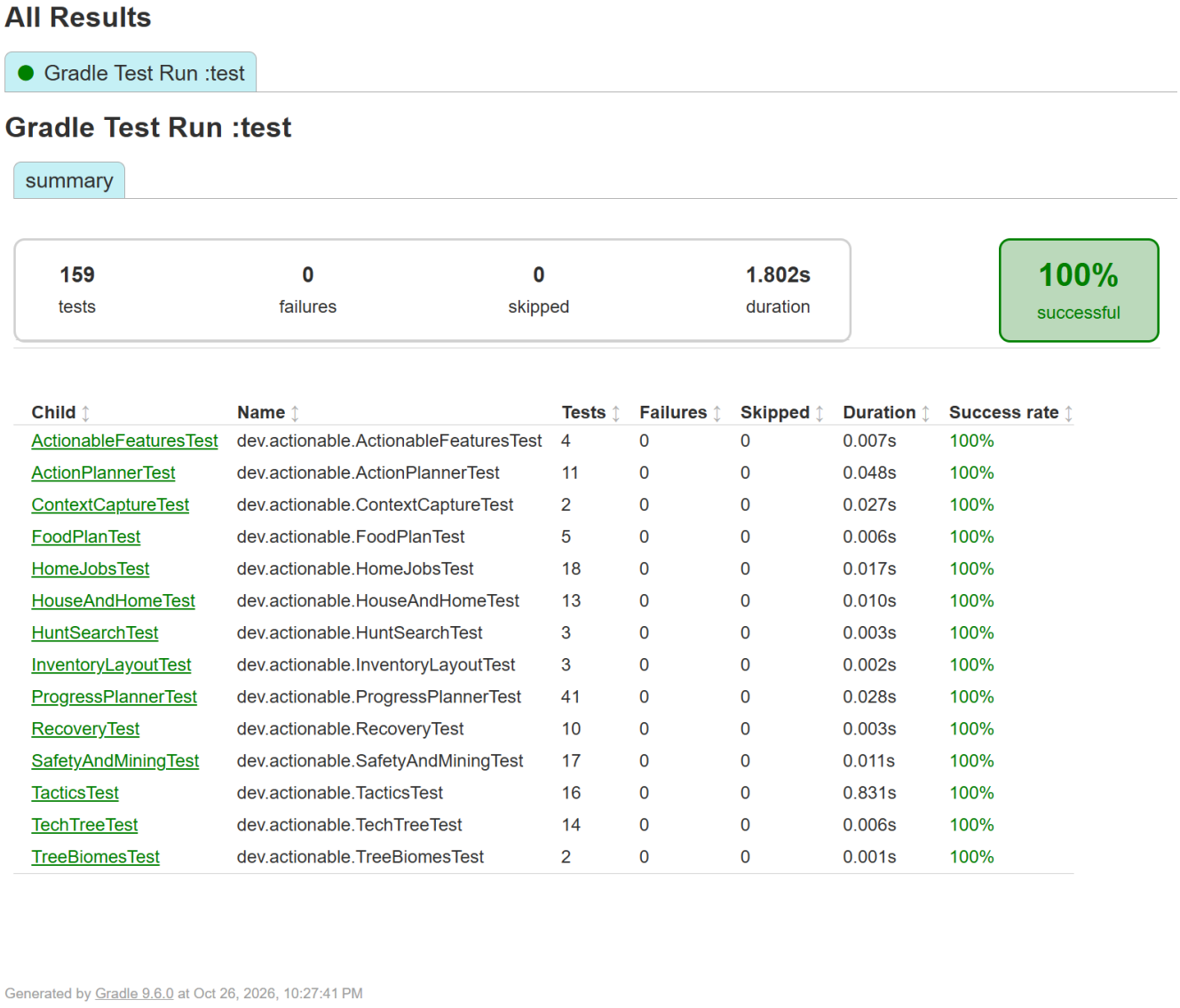
    Agent->>Mod: Scan entities within 16-block radius
    Mod->>Ollama: POST telemetry JSON (Async / target threat triage)

    alt Successful Inference (< 100ms)
        Ollama-->>Mod: Return structured JSON tactical decision
        Mod->>Baritone: Dispatch waypoint and navigation commands
        Mod->>MC: Execute shield raise, weapon cooldown strike, and movement
    else Alternate 2a: Ollama Server Timeout (> 100ms)
        Ollama--xMod: Connection timeout / no response
        Mod->>Mod: Catch timeout exception
        Mod->>Baritone: Cancel active pathing
        Mod->>MC: Force fallback defensive stance (raise shield & backpedal)
        Mod->>Mod: Log timeout warning to logs/gemma_agent.log
    end
```

Agent Check

I used Github Pro's Copilot (Using Claude Sonnet & GPT-6 interchangeably) to generate the additional features and their tests.

I could then run those tests with “./gradlew test”, and an HTML file with the results was generated in the build folder. Here is a screenshot of the results:



All Results

Gradle Test Run :test

Gradle Test Run :test

summary

159	0	0	1.802s	100%
tests	failures	skipped	duration	successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
ActionableFeaturesTest	dev.actionable.ActionableFeaturesTest	4	0	0	0.007s	100%
ActionPlannerTest	dev.actionable.ActionPlannerTest	11	0	0	0.048s	100%
ContextCaptureTest	dev.actionable.ContextCaptureTest	2	0	0	0.027s	100%
FoodPlanTest	dev.actionable.FoodPlanTest	5	0	0	0.006s	100%
HomeJobsTest	dev.actionable.HomeJobsTest	18	0	0	0.017s	100%
HouseAndHomeTest	dev.actionable.HouseAndHomeTest	13	0	0	0.010s	100%
HuntSearchTest	dev.actionable.HuntSearchTest	3	0	0	0.003s	100%
InventoryLayoutTest	dev.actionable.InventoryLayoutTest	3	0	0	0.002s	100%
ProgressPlannerTest	dev.actionable.ProgressPlannerTest	41	0	0	0.028s	100%
RecoveryTest	dev.actionable.RecoveryTest	10	0	0	0.003s	100%
SafetyAndMiningTest	dev.actionable.SafetyAndMiningTest	17	0	0	0.011s	100%
TacticsTest	dev.actionable.TacticsTest	16	0	0	0.831s	100%
TechTreeTest	dev.actionable.TechTreeTest	14	0	0	0.006s	100%
TreeBiomesTest	dev.actionable.TreeBiomesTest	2	0	0	0.001s	100%

Generated by Gradle 9.6.0 at Oct 26, 2026, 10:27:41 PM

My whole prompt was: Implement a new feature to bring the code up to these specs: (Followed by a paste of everything written starting at Part 1 #4 down to the end of User Stories).

While doing in-game testing, the one-prompt fix above created some behavior I had not accounted for. The autonomous player would completely freeze up and do effectively nothing while tracking a nearby mob that it chose to target. This would interrupt the current plan, forcing it to stop and track the mob, as if the mod was expecting the mob to attack. The issue was, however, that the mobs it was tracking were in caves beneath the surface, meaning they did not have line-of-sight that would have triggered a PvE attack. At the same time, the LLM

was unable to take direct control since the mod would force it into defense mode, but would not proceed to get closer to the threatening mob.

Mobs, when unprovoked and without line of sight, have two modes: wander and look-around. Wander does not bring them closer to a player, but moves about 5-10 blocks in any direction and has a slight bias to go climb while pathfinding rather continue deeper in the world. look-around is merely an animation, the mob will pause periodically to look around before wandering again. This behavior effectively locks the mod in a frozen loop where it waits for the “attacking” mob to approach. Killing these nearby mobs with commands would allow the LLM to continue its previous action.

This issue is not directly expressed in the requirements, nor is it tested for in the test suite.

STATUS: FIXED *(Added FR-14)*

Another thing it made up was this concerning line in the README:

```
`/llmagent config endpoint <url>` sets the Ollama URL (e.g. `http://localhost:11434`);
it is saved in `config/`.
```

This is concerning to me because it may have misinterpreted the fix, thinking it needed an entirely different LLM runtime, json parser, etc. to have a dedicated chat for combat events. While this idea is not immediately awful, it presents additional potential problems, such as doing planning on two different chat instances at once, slowing down both LLM runtimes. It may not be this, however. It could instead just be an additional feature I did not originally intend that allows for LAN ollama connections / other port configurations.

STATUS: LEFT OPEN *(Does not immediately cause issues, requires further investigation. Moved to Open Questions.)*